NovaCare Final Project Report

1. Project Overview

NovaCare is an integrated AI-powered healthcare companion and robotic assistant designed to empower independence for individuals with physical or sensory disabilities. The system combines modern web technologies, machine learning, and physical robotics (SERBot) to create a seamless, multimodal interaction experience.

The core mission of NovaCare is to bridge communication gaps (via Sign Language Recognition and Text-to-Speech) and provide intelligent assistance through Conversational AI and Emotion Detection.

2. System Architecture

The NovaCare repository is organized into a domain-driven, microservice-based architecture to ensure scalability, isolated development, and maintainability.

2.1 Codebase Structure

- `apps/`: End-user interfaces (Next.js Dashboard, Flutter Mobile App, React Robot UI).
- `services/`: Core backend microservices (ASL, LLM, Robot HAL, TTS, Optimized Edge Runtime).
- `shared/`: Common data models, database schemas, utility functions, and configurations.
- `infrastructure/`: Deployment scripts, Docker orchestration (`docker-compose.yml`), and database migrations.

---

3. American Sign Language (ASL) Recognition Subsystem

A major component of NovaCare is its real-time ASL recognition pipeline, which allows non-verbal users to communicate naturally with the robot.

3.1 Model Architecture

The ASL recognition model utilizes an **Attention-based Multilayer Perceptron (MLP)**. - **Input**: The system uses Google MediaPipe to extract 21 3D hand landmarks (63 continuous numerical features) per video frame. - **Attention Mechanism**: The network treats each of the 21 landmarks as a "token" and applies Multi-Head Self-Attention. This allows the model to learn the complex spatial relationships between different fingers (e.g., how the thumb relates to the index finger for specific letters). - **Output**: The network predicts across 29 classes (A-Z alphabet, Space, Delete, and an "Unknown/Idle" state).

3.2 Data and Training

The model was trained on a robust dataset of **25,479 real-world samples**. - **Preprocessing**: Landmarks are normalized relative to the wrist (landmark 0) to ensure the model is invariant to where the hand is located in the camera frame. The coordinates are scaled by the distance to the middle finger (landmark 9) to account for hands being closer or further from the camera. - **Augmentation**: During training, the landmarks undergo synthetic augmentations including random 3D rotations, scaling, translation, and Gaussian noise injection to prevent overfitting and improve real-world robustness. - **Performance**: The final trained model achieved a **91.59% validation accuracy** and **91.49% test accuracy** on held-out data.

3.3 Inference and Frontend Integration

- **Real-time Pipeline**: The Next.js frontend captures webcam frames and sends base64-encoded images to the FastAPI backend.
- **Temporal Stabilization**: To prevent noisy, flickering predictions, the system requires a letter to be continuously predicted with high confidence for **1.5 seconds** before it is "confirmed".
- **Accumulator**: Confirmed letters are added to a server-side text accumulator. The user can build a full word or sentence and send it to the Conversational LLM for a response.

---

4. Conversational AI and Emotion Detection (LLM) Subsystem

The Conversational AI module acts as the cognitive center of NovaCare, allowing natural language interaction and emotional intelligence.

4.1 Architecture and APIs

- **Framework**: Built on Flask (port 5000), exposing RESTful endpoints for chat, history management, and context updates.
- **LLM Engine**: Integrates with Large Language Models (LLMs) to process user prompts, maintain conversation history, and generate context-aware responses.
- **Prompt Engineering**: System prompts are carefully designed to instruct the AI to act as a empathetic, concise, and helpful medical companion.

4.2 Emotion Detection

- **Background Polling Architecture**: To avoid hardware locking conflicts between backend services and the frontend ASL camera, emotion detection was shifted to a silent frontend background poller. The Next.js frontend

uses a custom React hook (`useBackgroundEmotion`) to capture invisible frames every 5 seconds and evaluates them against a Hugging Face ViT model (`trpakov/vit-face-expression`).
- **Live Context Injection**: The frontend dynamically injects the latest detected emotion and confidence score directly into the `POST /api/chat` payload.
- **Empathy Engine**: The `MentalHealthOrchestrator` and Conversational AI explicitly ingest this non-verbal data into the LLM's Live Context window (RAG). If a user says "I am fine" but their facial expression reads "sad", the LLM is contextually aware of the contradiction and tailors its empathy and response tone accordingly.
- **State Emitting**: Based on the conversation and facial analysis, the LLM emits emotional states (e.g., `happy`, `concerned`, `thinking`) which are broadcasted to the Robot UI to physically change the robot's animated facial expressions, bridging the gap between digital AI and physical empathy.

---

5. Text-to-Speech (TTS) Subsystem

To provide a voice for NovaCare, the TTS subsystem converts the LLM's text responses into natural-sounding audio.

5.1 Architecture

- **Framework**: A standalone proxy service running on port 8002.
- **Edge-Optimized**: Designed to be lightweight enough to run on edge hardware without requiring massive local deep learning models or expensive cloud API calls.
- **Integration**: Works asynchronously within the Optimized Edge Runtime. When the LLM generates a response, it is immediately queued to the TTS service, which streams audio to the robot's physical speakers.

---

6. Robot Hardware Abstraction Layer (HAL)

The SERBot relies on the Robot HAL to abstract complex motor and hardware commands into simple, reliable REST APIs.

6.1 Architecture and Features

- **Framework**: Python-based service running on port 9000.
- **Motor Control**: Exposes endpoints for physical navigation (`move_forward`, `turn`, `stop`), abstracting away the low-level GPIO or serial commands required to drive the SERBot's motors.

- **Peripheral Management**: Handles LED controls and physical speaker output.
- **Safety**: Includes built-in safety checks to prevent hardware damage, providing a safe testing environment for high-level software.

---

7. The Optimized Edge Runtime

At the heart of the robot's physical operation is the **Optimized Runtime**, an asynchronous orchestrator that ties all microservices together on the physical robot.

7.1 Architecture

- **Async Orchestrator**: Built entirely around Python's `asyncio`, managing concurrent tasks without blocking the main thread.
- **Task Queues**: Implements priority-based task pipelines for audio capture, camera frames, and AI inference.
- **State Management**: Maintains a thread-safe, unified state of the robot's hardware (battery, CPU), current emotion, and conversation context.
- **WebSocket Communication**: Broadcasts real-time state updates at 10Hz to the Robot UI.
- **Non-Destructive Adapters**: Wraps the existing HTTP REST APIs (ASL, LLM, TTS, Robot HAL) into the async pipeline, acting as thin, fault-tolerant clients that do not modify the original microservices.

---

8. End-User Applications

8.1 Next.js Web Dashboard

- **Role**: The primary control center for guardians, medical professionals, and patients.
- **Features**: Real-time ASL recognition interface, vital sign monitoring, chat interface, and robot status dashboards.

8.2 Flutter Mobile App

- **Role**: A companion app for on-the-go access.
- **Features**: Push notifications, remote monitoring, and direct communication with the NovaCare system.

8.3 React Robot UI

- **Role**: The "face" of the SERBot, displayed on a physical screen mounted on the robot.

- **Features**: Animated eyes that change color and shape based on the current emotion state (e.g., happy blink, thinking rotation). It also includes audio level visualizations and connection indicators, entirely driven by the WebSocket stream from the Edge Runtime.

---

9. Deployment and DevOps

NovaCare is designed to be easily deployable both locally for development and physically onto the SERBot hardware.

9.1 Containerization and Scripts

- **Dockerization**: All microservices are containerized. There are separate Docker profiles (`Dockerfile.serbot`, `Dockerfile.laptop`) for edge deployment (optimized for Jetson/Raspberry Pi hardware constraints) and heavy laptop-based inference.
- **Unified Scripts**: The project features a unified set of PowerShell and Bash scripts (`start_all.ps1`, `deploy_serbot.sh`) that handle spinning up the entire microservice ecosystem, managing virtual environments, and deploying code to the physical robot via SSH/SCP.
- **Health Monitoring**: Automated scripts (`health_check.sh`) continuously verify the availability of all endpoints and ports.

---

10. The Team

NovaCare was brought to life by a dedicated team of engineers: - **Basant Awad** - **Nadira El-Sirafy** - **Noureen Yasser** - **Muhammad Mustafa** - **Ramez Asaad**

Generated: June 2026